Multirotor UAV Dynamics System Identification Manual

0 Overview: Five-Stage Identification Roadmap

Existing UAV model identification research suffers from four principal deficiencies: (1) absence of a validated propeller dynamic thrust model under real PWM and inflow conditions; (2) no unified formulation for fuselage aerodynamic drag or drag-moment models; (3) frequency-domain studies neglect CG offset, flight-controller tilt, and eccentricity moments; and (4) identified models are rarely applied to controller feasibility testing, fault simulation, or cluster-flight development.

This manual addresses all four deficiencies through a five-stage sequential pipeline:

Stage	Name	Core Objective	Primary Output	Dominant Technique
1	Propulsion Model ID	Static & dynamic characterization of the motor-propeller unit	PWM→ω, ω→T, ω→Q static polynomials; G_T(s), G_Q(s) transfer functions	Bench step + sine sweep; lsqcurvefit + spafdr/tfest
2	Drag Model ID	Identify aerodynamic drag and rotational damping coefficients	C_d_aero per flight axis; C_dp, C_dq, C_dr (damping); MOI	Bifilar pendulum; wind tunnel; real-flight freq-domain & force-balance
3	Model Correction	Align bench propulsion model with actual in-flight performance	Corrected linear thrust coefficients a_T, b_T	In-flight constrained least squares (lsqnonlin)
4	Time-Domain Validation	HITL simulation vs. real flight: velocity, maneuverability, PWM fidelity	Validated full 6-DOF time-domain model	HITL with MCC controller; PID comparison flights
5	Frequency-Domain ID	Attitude-loop open/closed-loop Bode characterization	Closed-loop TF model (3rd order typical, >80% fit)	Disturbance sweep flight + System ID Toolbox

Development principle: each stage builds strictly on the previous. Stages 1–3 feed the time-domain 6-DOF model; Stage 4 validates it via Hardware-In-The-Loop (HITL) tests; Stage 5 extends to frequency-domain characterization for advanced controller design.

Research scope limitations in existing literature (all addressed here):

- Static propulsion models only — no dynamic thrust characterization under real inflow
- Drag models assumed rather than identified — no wind tunnel or real-flight validation
- Frequency-domain identification ignores CG offset and flight-controller installation angle biases
- Identified models not validated for controller feasibility, fault simulation, or cluster flight

1 Time-Domain 6-DOF Model

1.1 Governing Newton-Euler Equations

Under the assumption of negligible gyroscopic effects (valid for general non-aggressive flight where the magnitude is too small to be considered), the 6-DOF dynamics of the UAV in head mode are described by the Newton-Euler equations. The translational dynamics along the three body-frame axes and the rotational dynamics about each axis are:

$$m \cdot \dot{v}_x = T \sin\theta - f_dx$$

$$m \cdot \dot{v}_y = T \sin\varphi \cos\theta - f_dy$$

$$m \cdot \dot{v}_z = T \cos\varphi \cos\theta - f_dz$$

$$J_x \cdot \dot{p} = \tau_p - \tau_dp$$

$$J_y \cdot \dot{q} = \tau_q - \tau_dq$$

$$J_z \cdot \dot{r} = \tau_r - \tau_dr \quad (1)$$

Variable definitions:

Symbol	Description	Unit
m	Total UAV mass	kg
v_x, v_y, v_z	Body-frame airspeeds: forward (head), lateral, vertical (≈ ground speed under light/no wind)	m/s
f_dx, f_dy, f_dz	Aerodynamic drag forces along each body axis (form to be identified)	N
φ, θ, ψ	Roll, pitch, yaw Euler angles	rad
J_x, J_y, J_z	Principal moments of inertia about roll, pitch, yaw axes	kg·m²
p, q, r	Roll, pitch, yaw angular rates	rad/s
τ_p, τ_q, τ_r	Aerodynamic control torques (from propellers)	N·m
τ_dp, τ_dq, τ_dr	Rotational aerodynamic drag moments (form to be identified)	N·m

1.2 Thrust and Torque Allocation — X-Type Quadrotor

For an X-type quadrotor (ignoring CG offset and flight-controller installation tilt), the total thrust T and the three-axis torques produced by the four propellers are:

$$T = C_T \cdot \Sigma_i \omega_i^2$$

$$\tau_p = (\sqrt{2} \cdot l/2)(-T_1 + T_2 + T_3 - T_4)$$

$$\tau_q = (\sqrt{2}\cdot l/2)(T_1 - T_2 + T_3 - T_4)$$

$$\tau_r = C_Q(\omega_1{}^2 + \omega_2{}^2 - \omega_3{}^2 - \omega_4{}^2) \quad (2)$$

where: C_T = propeller thrust coefficient [N·s²]; C_Q = reaction-torque coefficient [N·m·s²]; ω_i = rotational speed of i-th propeller [rpm]; T_i = thrust of i-th propeller [N]; l = arm length (half the inter-motor distance for co-axial pair) [m]. Motor numbering convention: 1=front-right, 2=rear-right, 3=rear-left, 4=front-left (CW: 1,3; CCW: 2,4).

1.3 Drag Model — Two Candidate Forms

The form of the drag and drag-moment terms is a key uncertainty to be resolved through identification. Two forms are commonly used in the literature:

Form A — Quadratic (aerodynamic) drag:

$$f_dx = C_dx \cdot v_x^2, \quad f_dy = C_dy \cdot v_y^2, \quad f_dz = C_dz \cdot v_z^2 \quad \tau_dp = C_dp \cdot p^2, \quad \tau_dq = C_dq \cdot q^2, \quad \tau_dr = C_dr \cdot r^2 \quad (3)$$

Form B — Viscous (linear) drag [confirmed applicable, §3]:

$$f_dx = C_dx \cdot v_x, \quad f_dy = C_dy \cdot v_y, \quad f_dz = C_dz \cdot v_z \quad \tau_dp = C_dp \cdot p, \quad \tau_dq = C_dq \cdot q, \quad \tau_dr = C_dr \cdot r \quad (4)$$

Section 3 presents identification results confirming that the viscous form (4) accurately describes FX-series UAV drag in normal flight regimes. The coefficients C_dx, C_dy, C_dz are identified independently per axis.

1.4 Key Identification Challenges

Three fundamental challenges complicate time-domain model identification:

1.4.1 Measurement Difficulty

The quantities T, f_d, τ_d, C_T, C_Q, and ω_i are all difficult to measure directly during flight. Carrying additional sensors increases mass, cost, and control latency. This manual resolves these challenges through a staged bench + flight identification approach.

1.4.2 Missing Model Components

The controller input to the UAV is a PWM signal, not rotational speed. Therefore, Equations (1–2) cannot be directly applied to controller verification without a PWM→ω motor model. Additionally, the effects of CG offset and flight-controller installation angle produce additional eccentricity moments not captured in the standard model.

1.4.3 Drag Form Ambiguity

Both quadratic (3) and viscous (4) forms appear in the literature, with different physical justifications. Identification experiments (§3) resolve this ambiguity empirically for the FX-series platform.

1.5 Complete Parameter List

Parameter Group	Symbols	Identification Method	Stage
Motor static model	c_w1, c_w2	Bench step test → lsqcurvefit	1
Propeller static model	c_T1, c_T2; c_Q1, c_Q2	Bench step test → lsqcurvefit	1
Motor dynamic TF	G_ω(s)	Bench sweep → two-stage spafdr/tfest	1
Propeller dynamic TF	G_T(s), G_Q(s)	Bench sweep → two-stage spafdr/tfest	1
Aerodynamic drag	C_dx, C_dy, C_dz	Wind tunnel + real-flight frequency-domain / force balance	2
Rotational damping	C_dp, C_dq, C_dr	Bifilar pendulum + flight log analysis	2
Moment of inertia	J_x, J_y, J_z	Bifilar pendulum (oscillation period method)	2
Structural	m, l	Direct measurement	2
Corrected thrust	a_T, b_T	In-flight constrained least squares	3
Corrected yaw torque	a_M	Static integrated torque model (Stage 1 result retained)	3

2 Stage 1: Propulsion Model Identification

2.1 Model Architecture and Taxonomy

The propulsion model maps the motor PWM command to the resulting propeller thrust and reaction torque. It is organized into a two-level hierarchy:

Level	Type	Input → Output	Model Form	Constraint
Static	Motor	PWM (μs) → ω (rpm)	Polynomial $c_{w1}\cdot u^2 + c_{w2}\cdot u$ − offset	ω = 0 at u = 1000 μs
Static	Propeller — Thrust	ω (rpm) → T_s (N)	Polynomial $c_{T1}\cdot\omega^2 + c_{T2}\cdot\omega$	T = 0 at ω = 0
Static	Propeller — Torque	ω (rpm) → Q_s (N·m)	Polynomial $c_{Q1}\cdot\omega^2 + c_{Q2}\cdot\omega$	Q = 0 at ω = 0
Static	Integrated PWM→T	PWM (μs) → T_s (N)	Cubic polynomial with offset constraint	T = 0 at u = 1000 μs
Dynamic	Motor TF	ω_s → ω_d	Transfer function G_ω(s)	Two-stage; ω_s = static-model output
Dynamic	Propeller Thrust TF	T_s → T_d	Transfer function G_T(s)	Two-stage; T_s = static-model output
Dynamic	Propeller Torque TF	Q_s → Q_d	Transfer function G_Q(s)	Two-stage; Q_s = static-model output

The two-stage dynamic approach is strongly preferred over direct identification (PWM →

dynamic output). Direct identification yields orders ≥10 or poor fit at 1 Hz; the two-stage approach using static output as the intermediate signal achieves substantially lower model order and higher goodness-of-fit.

2.2 Static Model Identification

2.2.1 Experimental Setup

Static propulsion measurements are obtained using a comprehensive power test bench (e.g., DET G10 10KGF). The bench measures PWM, rotational speed, thrust, and reaction torque simultaneously under steady-state (step) excitation.

Step test parameters:

Parameter	Value	Rationale
Initial PWM	1000 μs	Idle (no thrust)
End PWM	1900 μs	Near-maximum throttle
Step size	50 μs	Fine resolution across full range
Step duration	5 s	Allows steady-state settling
Data recording	Simplified / oversampled	Reduce file size; retain steady-state values

2.2.2 MATLAB Fitting Procedure

Step 1 — Data preparation:

```
PWM = PWMus;          % motor PWM input [μs]
w   = NRPM;           % motor / propeller speed [rpm]
T   = FKG * 9.8;      % thrust: kg → N
Q   = TNM;            % reaction torque [N·m]
% Intercept the rising step segment for fitting
PWMtest = PWM(1:1138);   Ttest = T(1:1138);
wtest   = w(1:1138);     Qtest = Q(1:1138);
```

Step 2 — Motor model (PWM → ω):

Constraint: ω = 0 when PWM = 1000 μs → function form includes a fixed offset:

$$f(c, u) = c_1 \cdot u^2 + c_2 \cdot u - (c_1 \cdot 1000^2 + c_2 \cdot 1000)$$

```
f  = @(c,x) c(1).*x.^2 + c(2).*x - c(1)*1000^2 - c(2)*1000;
[cw, ~] = lsqcurvefit(f, [0 0], PWMtest, wtest);
% Display: cw(1) = c_w1,  cw(2) = c_w2
```

Step 3 — Propeller models (ω → T and ω → Q):

Constraint: T = Q = 0 when ω = 0 → function form passes through origin:

$$f(c, \omega) = c_1 \cdot \omega^2 + c_2 \cdot \omega$$

```
fT = @(c,x) c(1).*x.^2 + c(2).*x;
[cT, ~] = lsqcurvefit(fT, [0 0], wtest, Ttest);
[cQ, ~] = lsqcurvefit(fT, [0 0], wtest, Qtest);
```

Step 4 — Integrated model (PWM → T directly):

```
fi = @(c,x) c(1).*x.^3 + c(2).*x.^2 + c(3).*x
           - (c(1)*1000^3 + c(2)*1000^2 + c(3)*1000);
[ct, ~] = lsqcurvefit(fi, [0 0 0], PWMtest, Ttest);  % same for Q
```

2.2.3 Identified Static Models — FX450 Example

Model	Identified Expression	Fit Quality
PWM → ω	$\omega = -6.586\times10^{-3}\cdot u^2 + 27.670\cdot u - 21083.7$ [rpm]	Excellent; nonlinear character at PWM 1100–1450
ω → T	$T_s = 3.340\times10^{-7}\cdot\omega^2 - 2.881\times10^{-4}\cdot\omega$ [N]	Near-perfect fit; confirms quadratic character
ω → Q	$Q_s = 7.120\times10^{-9}\cdot\omega^2 - 7.457\times10^{-6}\cdot\omega$ [N·m]	Near-perfect fit; smooth torque-speed curve
PWM → T (integrated)	$T_s = -4.082\times10^{-8}\cdot u^3 + 1.841\times10^{-4}\cdot u^2 - 0.2486\cdot u + 105.3$ [N]	Good; cubic captures nonlinear PWM saturation
PWM → Q (integrated)	$Q_s = -7.95\times10^{-10}\cdot u^3 + 3.619\times10^{-6}\cdot u^2 - 4.910\times10^{-3}\cdot u + 2.086$ [N·m]	Good

Conclusion: *Static models accurately represent steady-state propulsion characteristics. The decomposed (ω→T, ω→Q) form is physically interpretable and directly serves the dynamic model identification pipeline.*

2.3 Dynamic Model Identification

2.3.1 Principle and Two-Stage Architecture

The dynamic model quantifies the frequency-dependent deviation of actual thrust/speed from the static model prediction. The key insight is that using the static model output as the dynamic model input (two-stage) dramatically improves identification quality:

$$\text{Stage A:}\quad \omega_s = f_static(PWM) \quad or \quad T_s = f_static(\omega) \qquad (5)$$

$$\text{Stage B:}\quad G(s) = \omega_d(s)/\omega_s(s) \quad or \quad G(s) = T_d(s)/T_s(s) \qquad (6)$$

In contrast, the single-stage approach (e.g., G(s) = ω_d(s)/PWM(s)) requires orders ≥10 with poor high-frequency fit and is not recommended for simulation development.

2.3.2 Frequency Sweep Experiment

Dynamic data is obtained by applying a logarithmic sine sweep to the motor PWM input:

Parameter	Recommended Value	Note
Sweep waveform	Logarithmic sine sweep	Logarithmic frequency progression covers wide band evenly
Frequency range	0.2 – 20 Hz	Upper limit constrained by 30 s sweep duration
PWM range (mid)	1200 – 1900 μs	Covers most of the operational throttle range

Parameter	Recommended Value	Note
Sweep duration	30 s (max test bench limit)	Higher frequencies are identified with lower resolution
Sweep power	1.02	Controls frequency growth rate
Sampling rate	10,000 Hz	High-frequency data capture for full-spectrum analysis

Note: *The 30 s duration limits the maximum resolvable frequency. For identification above 1 Hz, the model fit quality degrades; the best validated bandwidth is 0–20 Hz for the thrust model using the two-stage approach.*

2.3.3 Full MATLAB Identification Procedure

Step 1 — Load sweep data and compute static outputs:

```
% Compute static outputs from measured inputs
ws = cw(1)*PWM.^2 + cw(2)*PWM - cw(1)*1000^2 - cw(2)*1000;  % static ω
Ts = cT(1)*w.^2 + cT(2)*w;   % static thrust
Qs = cQ(1)*w.^2 + cQ(2)*w;   % static torque
```

Step 2 — Low-pass filter all signals:

```
fs = 1e4;   lf = 5;    % sampling freq [Hz], filter cut-off [Hz]
PWMf = lowpass(PWM, lf, fs);
wf   = lowpass(w,   lf, fs);   wsf  = lowpass(ws,  lf, fs);
Tf   = lowpass(T,   lf, fs);   Tsf  = lowpass(Ts,  lf, fs);
Qf   = lowpass(Q,   lf, fs);   Qsf  = lowpass(Qs,  lf, fs);
```

Step 3 — Downsample to target identification frequency:

```
fd = 20 * lf;     % downsample to 20× target freq (here: 100 Hz)
wfd  = resample(wf,  fd, fs);   wsfd = resample(wsf, fd, fs);
Tfd  = resample(Tf,  fd, fs);   Tsfd = resample(Tsf, fd, fs);
Qfd  = resample(Qf,  fd, fs);   Qsfd = resample(Qsf, fd, fs);
```

Note: *Downsampling may distort data at the front and end of the record. Intercept the clean middle segment for identification. For equidistantly sampled data, use equidistant downsampling to preserve timing.*

Step 4 — Estimate empirical frequency response:

```
% Example: two-stage ω_s → ω_d dynamic identification
data = iddata(wfd, wsfd, 1/fd);
fvec = linspace(0, lf*2*pi, 500);   % or logspace for log-spaced
g    = spafdr(data, [], fvec);       % estimate empirical FRF
```

Step 5 — Fit and select best reduced-order TF:

```
for i = 0:10
    sys_i       = tfest(g, i);
    [~,fit(i+1),~] = compare(sys_i, data);
end
sys_best = tfest(g, find(fit==max(fit))-1);  % best-fit order
compare(sys_best, data);    % time-domain fit check
bode(g, idfrd(sys_best));   % Bode plot validation
```

2.3.4 Identified Dynamic Transfer Functions — All Models

All transfer functions below are identified from FX450 bench sweep data using the two-stage approach:

① Motor: ω_s → ω_d (two-stage, 2nd-order reduced)

$$\omega_d(s)/\omega_s(s) = (10.18s - 9.326) / (s^2 + 8.014s - 9.268) \quad [\text{goodness-of-fit: } 82.24\%] \quad (7)$$

> **Note:** *10th-order model exists but 2nd-order provides acceptable fit with greatly reduced complexity.*

② Propeller Thrust: T_s → T_d (two-stage, 4th-order, valid 0–20 Hz)

$$T_d(s)/T_s(s) = (0.1447s^4 + 79.97s^3 + 5381s^2 + 1.17\times10^6 s + 3.689\times10^6) / (s^4 + 132.2s^3 + 1.291\times10^4 s^2 + 1.303\times10^6 s + 3.847\times10^6) \quad [\text{fit: } 94.59\%] \quad (8)$$

This model accurately reproduces experimental dynamic thrust in the 0–20 Hz band and is recommended for HITL simulation.

③ Propeller Torque: Q_s → Q_d (static gain — model is near-constant)

$$Q_d(s)/Q_s(s) \approx 0.986 \quad [\text{static gain; fit: } \sim 68\% \text{ only}] \quad (9)$$

> **Note:** *The reaction-torque dynamic model is inherently complex. A pure static gain indicates the dynamic effects are primarily nonlinear, not well-captured by a linear TF. The yaw controller verification may therefore show limited accuracy.*

④ Integrated Model (PWM → T_d) — for reference / NOT recommended

$$T_d(s)/PWM(s) = [\text{10th-order, complex coefficients}] \quad [\text{fit: poor above 1 Hz}] \quad (10)$$

This model requires order ≥10 at 1 Hz due to the compounding of motor and propeller nonlinearities. It is not recommended for simulation or controller development.

> **Conclusion:** *The two-stage decomposed dynamic model (T_s→T_d, 4th-order) is the recommended propulsion model for HITL simulation. Goodness-of-fit reaches 94.59% over 0–20 Hz, sufficient for controller feasibility testing.*

3 Stage 2: Aerodynamic Drag and Drag-Moment Identification

3.1 Aerodynamic Drag Model

3.1.1 Model Formulation

The full aerodynamic drag model with angle-dependent coefficient and fixed reference area is:

$$F_d = ½ \cdot \rho \cdot A \cdot C_d_aero(\alpha, \beta) \cdot v^2 \quad (11)$$

where A is the fixed nominal reference area of the full UAV airframe (calibrated from 3D geometry at zero AoA/sideslip; constant for a given UAV model), and C_d_aero(α,β) captures all attitude-dependent drag variation.

For the hover-to-moderate-speed regime relevant to this study, the drag simplifies to the viscous form:

$$F_d = C_d \cdot v \quad (viscous / linear\ form) \quad (12)$$

This simplification is justified by the identification results in §3.1.4: both indoor and outdoor methods consistently yield a first-order transfer function model with a single drag coefficient pole.

The Advance Ratio effect (J = V/nD, where V = inflow speed, n = prop speed, D = prop diameter) is NOT embedded in the propulsion model. It is fully characterized by the drag submodel — specifically, thrust attenuation under inflow is modeled as an equivalent aerodynamic drag force in the body axis. Wind tunnel tests (§2.2.2 of the original manual) confirmed that under the same PWM input, thrust decreases monotonically with inflow velocity, with attenuation magnitude increasing with wind level.

3.1.2 Method 1 — Frequency-Domain Identification (Indoor)

During indoor level flight in head mode, the X-axis dynamics satisfy:

$$v_x(s) = [1/(s + C_dx)] \cdot u_x(s) \quad (13)$$

where u_x is the X-axis control force (computed from pitch angle and thrust) and C_dx is the drag coefficient. Identification principle: fit a 1-pole / 0-zero continuous transfer function to the de-meaned input-output data.

Data preparation:

```
% Compute X-axis control force and de-mean to remove CG/installation bias
ux_raw = m*g * sin(pitch/g) ./ (cos(roll).*cos(pitch));
ux          =  datapreproc(ux_raw,  0.05,  'rmean',  true,  'rtrend',  false,
'tap_width', 0);
vx_out = vx;    % X-axis velocity (output)
% Sample time: 0.05 s (20 Hz).  Use System Identification Toolbox:
%  → Time domain data; 1 pole, 0 zeros, continuous TF; click Estimate
```

Identified model (FX200, indoor):

$$v_x(s)/u_x(s) = 0.7689 / (s + 0.4021) \rightarrow C_dx = 0.4021 \quad (14)$$

3.1.3 Method 2 — Force Balance (Outdoor, Multiple Speeds)

During uniform level flight at constant velocity, Newton's second law gives zero net

acceleration:

$$m \cdot a_x = u_x - F_dx = 0 \;\rightarrow\; F_dx = u_x = T \cdot \sin\theta \quad (15)$$

By flying at five speeds (2, 4, 6, 8, 10 m/s) and recording the mean control power at each speed, a speed-drag scatter plot is obtained. The drag coefficient is the slope of the best-fit line.

MATLAB code:

```
vx_vals = [2, 4, 6, 10];        % m/s (8 m/s excluded: excessive wind disturbance)
ux_vals = [mean_u2, mean_u4, mean_u6, mean_u10];     % de-meaned control forces

f  = @(c, x) c(1).*x + c(2);   c0 = [0 0];
[ct, ~] = lsqcurvefit(f, c0, vx_vals, ux_vals);
C_dx = ct(1);   % drag coefficient  (slope)
% FX200 outdoor result: C_dx ≈ 0.4263
```

Comparison of identified drag coefficients:

Method	Experiment Condition	Identified C_dx	Notes
Freq. Domain — Indoor	Calm indoor, step velocity±0.5 m/s	0.4021	High-quality data; recommended baseline
Force Balance — Outdoor	5 m/s crosswind; 2–10 m/s test speeds	0.4263	~6% deviation from indoor; wind disturbance
Wind Tunnel	Pitch AoA 0.1 rad; 8 wind levels	0.2187	Qualitative reference only; uneven inflow
Wind Tunnel	Pitch AoA 0.2 rad	0.1787	Decreases with AoA (increasing lift component)
Wind Tunnel	Pitch AoA 0.3 rad	0.1816	Consistent with 0.2 rad measurement

Conclusion: *Indoor frequency-domain identification (C_dx = 0.40) and outdoor force-balance identification (C_dx = 0.43) yield consistent results, confirming the viscous drag model form. The indoor result is recommended as the primary model parameter due to lower disturbance.*

3.2 Rotational Drag-Moment Identification

3.2.1 Physical Model

Assuming viscous rotational damping (valid for non-aggressive maneuvers), the roll angular velocity under free air damping decays as:

$$\dot{p} = -C_dp \cdot p \;\rightarrow\; p(t) = p_0 \cdot e\text{^}(-C_dp \cdot t) \quad (16)$$

Identification: measure the free angular velocity decay of the suspended UAV and extract the exponential decay constant C_dp. Pitch and yaw are treated identically.

3.2.2 Bifilar Pendulum Method

The UAV is suspended on two inelastic strings (minimum length 3 m, approximately 3–5× wheelbase), attached symmetrically about the CG. The device is rotated ~40° about the plumb vertical line of the CG and released. The flight controller records the free oscillation.

Three independent setups are required for the three axes (roll: strings in Y-direction; pitch: strings in X-direction; yaw: strings directly below CG with horizontal separation).

Envelope extraction and damping identification:

```
p_env = abs(hilbert(p));        % upper envelope via Hilbert transform
% Alternative: findpeaks(p) to extract peak values

p_ln  = log(p_env);             % ln transform → linear decay curve
% Each free-swing segment shows constant negative slope = -C_dp

C_dp  = diff(p_ln) * fs;        % numerical derivative
% Or fit linear function: p_ln = -C_dp*t + const
% C_dp = -slope

% Average across 3-5 repeat experiments
C_dp_final = mean(C_dp_segments);
```

Damping coefficient formula (from two time points):

$$C_dp = [\ln(y_2/y_1)] / (t_2 - t_1) \quad (17)$$

3.2.3 Identified Results

FX150 UAV — rotational damping coefficients (averages across 5 experimental groups):

Axis	Roll C_dp	Pitch C_dq	Yaw C_dr
Coefficient (s^{-1})	0.0064	0.0128	0.0120

Note: *These values are small relative to typical flight dynamics. In simulation models, setting all damping coefficients to zero introduces negligible error for non-aggressive flight scenarios. The values are included for physical completeness.*

3.3 Moment of Inertia via Bifilar Pendulum

3.3.1 Calculation Formula

The principal moments of inertia are computed directly from the oscillation period of the bifilar pendulum:

$$I = (m \cdot g \cdot d^2 \cdot T_n^2) / (16\pi^2 \cdot L) \quad (18)$$

where d = distance between the two strings [m]; L = string length [m]; T_n = simple harmonic oscillation period [s]; m = UAV mass [kg]; g = 9.8 m/s².

3.3.2 Period Calculation via FFT

```
fft_p  = fft(p_rollrate);    n = length(p_rollrate);    fs_log = 20;
fftsh  = fftshift(fft_p);
f_axis = linspace(-fs_log/2, fs_log/2, n);
fn_x    = max(f_axis(abs(fftsh)  == max(abs(fftsh))));   % dominant frequency
[Hz]
Tn_x   = 1 / fn_x;   % oscillation period [s]
% Repeat with pitch and yaw angular rates for Tn_y, Tn_z
```

3.3.3 Identified MOI — FX450

Axis	String Spacing d (m)	String Length L (m)	Period T_n (s)	MOI (kg·m²)
X (Roll)	0.385	1.02	0.9873	0.01850
Y (Pitch)	0.390	1.02	1.0209	0.02030
Z (Yaw)	0.565	1.02	0.8879	0.03223

UAV mass used: m = 2.015 kg (average of multiple measurements). All parameters directly measured; no estimation required.

4 Stage 3: Dynamic Model Correction

4.1 Motivation and Observed Deficiencies

The bench-measured static propulsion model exhibits three key deficiencies when applied to HITL simulation:

Deficiency	Symptom	Root Cause
Oscillation	HITL simulation causes UAV oscillations not seen in real flight	Static model does not capture dynamic propulsion characteristics under flight conditions
Inflow neglect	Model does not account for advance ratio effects under forward flight speed	Bench test conducted at zero airspeed — inflow reduces effective thrust
Control-effort deviation	Predicted PWM significantly exceeds or falls below real-flight values	Bench-to-flight performance gap; possibly includes ESC and battery dynamics

4.2 Corrected Model Formulation

The corrected model replaces the nonlinear bench propulsion model with a simple linear approximation identified from real flight data, while retaining all other model components:

$$T_i = a_T \cdot \sigma_i + b_T \quad \textit{(single propeller thrust, linear in normalized PWM)} \tag{19}$$

$Q_i = a_M \cdot \sigma_i$ *(reaction torque, zero intercept assumed)* (20)

The full corrected 6-DOF model (combining Eq.(1), Eq.(4), and Eq.(19)):

$$m\cdot\dot{v}_x = (a_T\cdot\Sigma\sigma_i + 4b_T)\cdot\sin\theta - C_dx\cdot v_x \quad m\cdot\dot{v}_y = (a_T\cdot\Sigma\sigma_i + 4b_T)\cdot\sin\varphi\cos\theta - C_dy\cdot v_y$$
$$m\cdot\dot{v}_z = (a_T\cdot\Sigma\sigma_i + 4b_T)\cdot\cos\varphi\cos\theta - C_dz\cdot v_z \quad J_x\cdot\dot{p} = \tau_p - C_dp\cdot p \quad J_y\cdot\dot{q} = \tau_q - C_dq\cdot q$$
$$J_z\cdot\dot{r} = \tau_r - C_dr\cdot r \quad (21)$$

where torques τ_p, τ_q, τ_r are computed via Eq.(2) using individual propeller thrusts $T_i = a_T\cdot\sigma_i + b_T$, and $\tau_r = a_M\cdot(\sigma_1 + \sigma_2 - \sigma_3 - \sigma_4)$.

4.3 Identification Conditions and Physical Constraints

Two prerequisite conditions must be satisfied during the identification flight:

Condition A (Attitude quality): Pitch tracking must be accurate with negligible steady-state error. This ensures that the attitude angle used in force computation is reliable.

Condition B (Altitude stability): Altitude must remain stable with minimal oscillation during the identification segment. This ensures the vertical force balance equation is valid.

These conditions are achieved using the MCC controller, which provides superior disturbance rejection compared to standard PID.

4.4 Optimization Problem Formulation

From the X-axis force balance during level flight at constant velocity:

$$(a_T\cdot\Sigma\sigma_i + 4b_T)\cdot\sin\theta - u_x = 0 \quad (22)$$

From the vertical force balance during stable altitude hold:

$$(a_T\cdot\Sigma\sigma_i + 4b_T)\cdot\cos\theta\cdot\cos\varphi - m\cdot g = 0 \quad (23)$$

Define residual functions:

$$f_X(a_T, b_T) = \|(a_T\cdot\Sigma\sigma_i + 4b_T)\cdot\sin\theta_m - u_x\|_2^{\,2} \quad (24)$$

$$f_Z(a_T, b_T) = \|(a_T\cdot\Sigma\sigma_i + 4b_T)\cdot\cos\theta_m\cdot\cos\varphi_m - m\cdot g\|_2^{\,2} \quad (25)$$

Combined constrained optimization:

$$\min\ [f_X; f_Z] \quad \text{subject to: } a_T > 0,\ -5 < b_T < 5 \quad (26)$$

4.4.1 MATLAB Implementation

```
% Build data vectors from level-flight log
X  = ux_ctrl(1340:3600);            % X-axis control output [N-equivalent]
Y  = sin(pitch(1340:3600));         % sin(θ)
Z  = -m*g*sin(ux./g)./(cos(roll).*cos(pitch)); % Xf-axis control force
K  = (pwm_sum(1140:1340) - 4000)/1000;         % normalized total PWM
J  = cos(roll(1140:1340)).*cos(pitch(1140:1340));
```

```
fun = @(x) [abs(Z - (scale./x(1).*X - 4*x(2)).*Y);
            abs(1/(4*x(1)).*K - 4*x(2) - m*g./J)];
x0 = [1, 25];   lb = [0, -inf];   ub_c = [inf, inf];
opts = optimoptions('lsqnonlin', 'Algorithm', 'trust-region-reflective');
x_sol = lsqnonlin(fun, x0, lb, ub_c, opts);
a_T = x_sol(1);   b_T = x_sol(2);
% Verify: check pitch tracking quality (Fig. 81 equivalent)
% Verify: check altitude stability (Fig. 82 equivalent)
```

4.4.2 Identified Result — FX200

$$T_i = 3.847 \cdot \sigma_i + 1.4592 \quad [N] \qquad (27)$$

where σ_i is the normalized PWM input. Note: slight variation between flights/days is normal due to ground effect, wind, and battery state. The reaction-torque model (τ_r) retains the static integrated model from Stage 1.

> ⚠ *The reaction-torque model cannot be corrected via this approach (a_M cancels out in the optimization). Use the Stage 1 static integrated model for Q. This limits yaw-control accuracy in HITL simulation.*
>
> **Conclusion:** *The corrected linear thrust model (T_i = 3.847·σ_i + 1.459) eliminates the bench-to-flight performance gap and significantly improves HITL simulation fidelity in all three axes.*

5 Stage 4: Time-Domain Model Validation

5.1 Validation Criteria

Model fidelity is assessed through three hierarchical criteria, progressing from surface behavior to fundamental actuator output:

Criterion	What is Compared	Observable Metric	Standard
Maneuverability	Subjective flight feel: HITL vs. real	Pilot impression during identical maneuvers	Similar control response feel; no significant difference
Controllability	Velocity step tracking: sim vs. real	Speed time series (v_x, v_y, v_z)	Similar overshoot %, settling time, and steady-state error
Fidelity (Core)	Actuator output: HITL PWM vs. real PWM	PWM time series at each motor	PWM magnitude and trend match within acceptable tolerance

PWM is the most fundamental metric: it directly reflects the force the model predicts must be applied to achieve a given state. Close PWM agreement between HITL and real flight validates the model at the physics level, not just the output level.

5.2 MCC Controller Validation — FX200

5.2.1 Step Velocity Test

Indoor step velocity flight (±0.5 m/s in X and Y, with Z altitude hold) was performed in both real flight and HITL simulation with identical MCC control parameters.

Metric	Real Flight	HITL	Assessment
X-velocity overshoot	~15–20%	~12–18%	✓ Consistent; small simulation advantage
X-velocity settling time	~2–3 s	~2–3 s	✓ Match
Z-velocity control	Stable; small ground-effect fluctuation	Stable; no ground-effect model	✓ Acceptable deviation
X-axis PWM level	~1300 μs range	~1300 μs range	✓ Core fidelity confirmed
Z-axis PWM trend	Consistent with hover	Matches trend; minor ground-effect deviation	✓ Pass

5.2.2 Random Velocity Test

Random multidirectional velocity commands (~15 direction changes) were applied. The three-axis velocity response of HITL simulation closely matched real-flight response, with consistent overshoot and convergence behavior across all axes.

Axis	Real-Flight Behavior	HITL Simulation	Match?
X (forward)	Correct step response shape	Near-identical amplitude and timing	✓
Y (lateral)	Correct step response shape	Near-identical amplitude and timing	✓
Z (vertical)	Stable altitude; minor fluctuation	Stable; model lacks ground-effect	✓ Acceptable

5.3 PID Controller Validation — FX200 Outdoor

5.3.1 Well-Tuned PID Parameters

Real outdoor flight and HITL simulation were compared with tuned PX4 PID parameters (max forward speed 1 m/s, angular rate limit 30 deg/s).

Metric	Real Flight	HITL	Assessment
X/Y velocity tracking	Good; affected by ~5 m/s crosswind	Slightly better (no wind disturbance model)	✓ Consistent core behavior
Yaw rate tracking	Good; minor steady-state error	Near-identical; slightly slower response	✓ Acceptable (torque model limitation)
PWM output level	~1300 μs; scattered (CG/installation tilt)	~1300 μs; concentrated (no CG model)	✓ Magnitude confirmed; spread difference explained

5.3.2 Mistuned PID (Oscillatory Flight)

PID parameters were deliberately modified to induce flight oscillations. Both real and HITL simulation exhibited:

- Pitch angle oscillations in range 0.02–0.04 rad (identical amplitude band in both)
- PWM oscillation: real flight 1200–1600 μs; HITL 1000–1800 μs (broader range in HITL — attributed to linear model approximation at PWM extremes)

Conclusion: *The corrected time-domain model successfully reproduces both normal and oscillatory flight behaviors under different controller parameter sets. HITL fidelity is sufficient for controller feasibility testing and development — the primary application target of this model.*

6 Stage 5: Frequency-Domain Model Identification

6.1 Motivation and Method Selection

While the time-domain model enables controller feasibility testing, frequency-domain models are required for: (1) evaluating controller bandwidth and stability margins; (2) designing frequency-domain controllers (H∞, loop-shaping); and (3) characterizing disturbance response (e.g., aerodynamic wake in multi-UAV flight).

Two excitation methods are compared:

Method	Mechanism	Advantages	Limitations
Reference Sweep	Inject sweep into position/velocity/attitude reference input	No controller modification needed	Risk of loss of control if bandwidth insufficient; limits frequency range to controller bandwidth
Disturbance Sweep (adopted)	Superimpose sweep signal onto actuator control output	Directly excites plant input; controller suppresses motion → safe wide-band sweep	Requires high-rejection controller (MCC); indoor space limits sweep duration

The disturbance sweep method was selected, exploiting the MCC controller's active disturbance rejection to enable safe indoor sweeps up to 100 Hz.

6.2 Sweep Signal Design

The sweep disturbance is a logarithmic sine signal superimposed on the controller output:

$$d(t) = A(t) \cdot \sin[2\pi \cdot f_0 \cdot (k^t - 1) / \ln k] \quad \text{for } t \in [0, T_max] \qquad (28)$$

where A(t) = amplitude envelope (may decrease with time to limit high-frequency excitation), f_0 = initial frequency, k = frequency growth factor, T_max = sweep duration.

FX200 roll channel sweep parameters: A = 0.05, f_0 = 0.01 Hz, k = 1.08, T_max = 80 s.

```
function rol_out = fcn(ch7, rol_ctrl, t)
  if (ch7 > 1300) && (t > 0) && (t <= 80)
    % Logarithmic sweep; amplitude constant at 0.05
    rol_out = 0.05 * sin((2*0.01*(1.08^t - 1)*pi) / log(1.08)) + rol_ctrl;
  else
    rol_out = rol_ctrl;
  end
end
```

```
% Roll disturbance is added to the MCC controller output BEFORE the mixer
```

Note: *Amplitude A should be tuned so the UAV exhibits visible attitude motion at low frequencies (≥ 10° swing) while staying within safe physical boundaries at high frequencies (< 3°). If the UAV approaches a boundary during sweep, abort via Ch5 Kill.*

6.3 Open-Loop Frequency-Domain Identification

6.3.1 Identification Principle and Setup

The open-loop model maps the combined actuator input (control + disturbance) to the resulting attitude state:

$$\text{Open-loop model: } \theta(s) / [u_ctrl(s) + u_sweep(s)] = G_OL(s) \quad (29)$$

Input: total actuator signal = e2 (combined control + sweep output). Output: pitch angle θ (e7) or pitch rate q. Data sampled at 250 Hz; System ID Toolbox used with low-pass filter at 35 rad/s cutoff.

6.3.2 Result and Assessment

Sweeping model orders from 1 to 5 poles: best fit = 43.51% at 5th-order model.

⚠ *Open-loop model fit quality (43.51%) falls below acceptable accuracy standards for simulation use. The open-loop TF model identified by this method is NOT recommended for simulation or controller design.*

Cause: the disturbance sweep signal is filtered by the closed-loop controller response before reaching the plant, reducing the effective signal-to-noise ratio at intermediate frequencies.

6.4 Closed-Loop Frequency-Domain Identification

6.4.1 Closed-Loop Identification Principle

From closed-loop control theory with unity feedback (H(s) ≈ 1) and reference R(s) = 0:

$$X(s) = G(s)/(1+C(s)G(s)) \cdot N(s) \;\rightarrow\; O(s) = E(s)\cdot C(s) = -C(s)\cdot X(s) \quad (30)$$

Rearranging:

$$X(s)/R(s) = C(s)G(s)/(1+C(s)G(s)) = -O(s)/X(s) \cdot X(s)/N(s) = -O(s)/N(s) \quad (31)$$

Therefore: identifying the reference-to-output closed-loop TF is equivalent to identifying the negative of disturbance-to-control-output:

$$\text{Closed-loop TF} = -Output_ctrl(s) / Disturbance_sweep(s) \quad (32)$$

6.4.2 Input/Output Definition for Identification

```
% Construct model input and output from sweep disturbance flight log
in  = e2 - ea4*tauy2sigma;   % total disturbance input = sweep excitation
ou  = -ea4*tauy2sigma;        % negative control output
% e2:  total actuator input (logged from mixer)
```

```
% ea4: controller output
% tauy2sigma: MCC scaling parameter (from controller file)

% System ID Toolbox settings:
% Sample time: 0.004 s (250 Hz)
% LP filter cut-off: 30 Hz
% Select data range: entire sweep duration
% Fit TF models: 1-5 poles; select min-order with best fit
```

6.4.3 Identified Closed-Loop Model — FX200 Pitch Channel

$$\theta(s)/\theta_ref(s) = (0.08012s^3 - 10.43s^2 + 1056s + 9234) / (s^3 + 34.16s^2 + 820.9s + 9635) \quad (33)$$

Model order: 3rd-order. Goodness-of-fit: 85.49%. Valid frequency range: up to ~30 Hz.

6.4.4 Validation with Independent Speed-Sweep

The closed-loop model is validated using an independent experiment: a chirp velocity reference (not a disturbance sweep) is applied, and the actual pitch response is compared with the model prediction.

```
% Validation experiment input/output
in_val = e4;    % pitch angle reference (from velocity sweep experiment)
ou_val = e7;    % actual measured pitch angle
% Load identified model → drag to 'Validation Data' → check 'Model output'
% Expected: high visual agreement between model and measured response
```

Validation result: the identified 3rd-order model closely tracks the actual pitch response throughout the independent validation sweep, confirming its predictive validity.

Conclusion: *The 3rd-order closed-loop pitch TF model (85.49% fit) is validated against independent data and is recommended for frequency-domain controller design, performance evaluation, and bandwidth analysis of the FX200 attitude loop.*

7 Symbol and Notation Reference

Symbol	Definition	Unit	Section
m	Total UAV mass	kg	§1.1
v_x, v_y, v_z	Body-frame airspeeds: forward, lateral, vertical (head mode)	m/s	§1.1
f_dx, f_dy, f_dz	Aerodynamic drag forces along body axes	N	§1.1
φ, θ, ψ	Roll, pitch, yaw Euler angles	rad	§1.1
J_x, J_y, J_z	Principal moments of inertia	kg·m²	§1.1
p, q, r	Roll, pitch, yaw angular rates	rad/s	§1.1
τ_p, τ_q, τ_r	Aerodynamic control torques from propellers	N·m	§1.1

Symbol	Definition	Unit	Section
τ_dp, τ_dq, τ_dr	Rotational aerodynamic drag moments	N·m	§1.1
T	Total thrust from all propellers	N	§1.2
T_i	Thrust of i-th individual propeller	N	§1.2
ω_i	Rotational speed of i-th motor/propeller	rpm	§1.2
C_T	Propeller thrust coefficient (quadratic)	N·s²/rpm²	§1.2
C_Q	Propeller reaction-torque coefficient	N·m·s²/rpm²	§1.2
l	Arm length (half inter-motor distance, co-axial pair)	m	§1.2
σ_i (or u_t)	PWM command to i-th motor	μs	§1.2
C_dx, C_dy, C_dz	Viscous aerodynamic drag coefficients (3 flight axes)	N·s/m	§3.1
C_dp, C_dq, C_dr	Rotational viscous damping coefficients (3 axes)	s^{-1}	§3.2
C_d_aero(α,β)	Attitude-dependent aerodynamic drag coefficient	dimensionless	§3.1
A	Fixed nominal reference area of full airframe	m²	§3.1
J = V/(nD)	Advance ratio (inflow velocity effect)	–	§3.1
ω_s, ω_d	Static model output and actual dynamic rotational speed	rpm	§2.3
T_s, T_d	Static model and actual dynamic propeller thrust	N	§2.3
Q_s, Q_d	Static model and actual dynamic reaction torque	N·m	§2.3
G_ω(s)	Dynamic TF: static speed → dynamic speed	–	§2.3, Eq.(7)
G_T(s)	Dynamic TF: static thrust → dynamic thrust	–	§2.3, Eq.(8)
G_Q(s)	Dynamic TF: static torque → dynamic torque	–	§2.3, Eq.(9)
c_w1, c_w2	Motor static model polynomial coefficients (PWM→ω)	rpm/μs², rpm/μs	§2.2
c_T1, c_T2	Propeller thrust polynomial coefficients (ω→T)	N·s²/rpm², N·s/rpm	§2.2
c_Q1, c_Q2	Propeller torque polynomial coefficients (ω→Q)	N·m·s²/rpm², N·m·s/rpm	§2.2
a_T, b_T	Corrected linear thrust model: slope and intercept	N/μs, N	§4.2, Eq.(19)
a_M	Reaction-torque model slope (intercept assumed zero)	N·m/μs	§4.2, Eq.(20)

Symbol	Definition	Unit	Section
d	String spacing in bifilar pendulum	m	§3.3, Eq.(18)
L	String length in bifilar pendulum	m	§3.3, Eq.(18)
T_n	Simple harmonic oscillation period	s	§3.3, Eq.(18)
f_s	Raw data sampling frequency	Hz	§2.3
f_d	Downsampled identification frequency	Hz	§2.3
lf	Low-pass filter cut-off frequency	Hz	§2.3

8 Quick-Reference Identification Checklist

This checklist tracks completion of all identification tasks. Each row references the relevant section. Complete tasks in order — later stages depend on earlier results.

Stage	ID	Task Description	Reference Section	Status
Stage 1 Propulsion	1.1	Bench step test: PWM 1000→1900 μs, 50 μs steps, 5 s/step	§2.2.1	☐
	1.2	Verify propeller rotation direction before recording data	§2.2.1	☐
	1.3	Fit Motor static model: PWM → ω (lsqcurvefit, quadratic)	§2.2.2	☐
	1.4	Fit Propeller static model: ω → T (lsqcurvefit, quadratic)	§2.2.2	☐
	1.5	Fit Propeller static model: ω → Q (lsqcurvefit, quadratic)	§2.2.2	☐
	1.6	Fit Integrated static model: PWM → T (cubic, optional)	§2.2.2	☐
	1.7	Conduct frequency sweep: 0.2–20 Hz, PWM 1200–1900 μs, 30 s	§2.3.2	☐
	1.8	Compute static model outputs from sweep data (T_s, Q_s, ω_s)	§2.3.3	☐
	1.9	Low-pass filter all signals (cut-off = target freq upper bound)	§2.3.3	☐
	1.10	Downsample to fd = 20 × f_max	§2.3.3	☐
	1.11	Estimate empirical FRF (spafdr)	§2.3.3	☐
	1.12	Fit dynamic TFs (tfest, iterate orders 0–10)	§2.3.3	☐
	1.13	Select best-fit order; attempt order reduction	§2.3.4	☐
	1.14	Validate all dynamic TFs with Bode plot	§2.3.4	☐

Stage	ID	Task Description	Reference Section	Status
		comparison		
Stage 2 Drag	2.1	Set up bifilar pendulum: ≥3 m strings, symmetric CG attachment	§3.2.2	☐
	2.2	Configure flight controller to log angular velocity at 20 Hz	§3.2.2	☐
	2.3	Rotate UAV ~40°, release, record free oscillation (repeat 3–5×/axis)	§3.2.2	☐
	2.4	Extract envelope: p_env = abs(hilbert(p))	§3.2.3	☐
	2.5	Apply ln transform; measure slope per segment → C_dp, C_dq, C_dr	§3.2.3	☐
	2.6	Compute oscillation period via FFT for roll, pitch, yaw	§3.3.2	☐
	2.7	Compute MOI (J_x, J_y, J_z) via bifilar formula Eq.(18)	§3.3	☐
	2.8	Indoor level-flight drag ID: de-mean data, fit 1-pole TF	§3.1.2	☐
	2.9	Outdoor drag ID: multiple-speed (2–10 m/s) force-balance method	§3.1.3	☐
	2.10	Compare indoor and outdoor C_d results for consistency	§3.1.3	☐
Stage 3 Correction	3.1	Record indoor/outdoor level-flight data (step + constant velocity)	§4.4	☐
	3.2	Verify pitch tracking quality (Condition A) from flight log	§4.3	☐
	3.3	Verify altitude stability (Condition B) from flight log	§4.3	☐
	3.4	Extract u_x, θ, φ, total PWM from log; build optimization data	§4.4.1	☐
	3.5	Solve constrained optimization (lsqnonlin) for a_T, b_T	§4.4.1	☐
	3.6	Record identified result: $T_i = a_T \cdot \sigma_i + b_T$	§4.4.2	☐
Stage 4 Validation	4.1	Implement corrected model in HITL (MCC controller firmware)	§5.1	☐
	4.2	HITL step-velocity test: compare v_x response vs. real flight	§5.2	☐
	4.3	HITL random-velocity test: compare 3-axis velocity response	§5.2	☐
	4.4	Compare HITL PWM output vs. real-flight PWM (core fidelity)	§5.2	☐
	4.5	PID validation: tuned parameters — outdoor flight vs. HITL	§5.3	☐
	4.6	PID validation: mistuned parameters — oscillatory flight vs. HITL	§5.3	☐
Stage 5 Freq-Domain	5.1	Design disturbance sweep: select A, f_0 , k, T_max per axis	§6.2	☐

Stage	ID	Task Description	Reference Section	Status
	5.2	Compile and flash MCC firmware with sweep injection	§6.2	☐
	5.3	Perform indoor disturbance sweep (roll, then pitch, then yaw)	§6.2	☐
	5.4	Attempt open-loop TF ID (note: expected poor fit)	§6.3	☐
	5.5	Perform closed-loop ID: define in/out per Eq.(32)	§6.4	☐
	5.6	Fit closed-loop TF (1–5 poles); select min-order model with fit > 80%	§6.4	☐
	5.7	Validate closed-loop model with independent speed-sweep data	§6.4.4	☐

Appendix A Complete Identified Model Summary

This appendix consolidates all identified model parameters for the FX-series UAV. All values are from experiments described in Sections 2–6. Use these tables as a quick reference when building or verifying simulation models.

A.1 Static Propulsion Model Parameters — FX450

Model	Coefficient	Identified Value	Unit
PWM → ω	c_w1	-6.586×10^{-3}	rpm/μs²
PWM → ω	c_w2	27.670	rpm/μs
ω → T (propeller)	c_T1	3.340×10^{-7}	N·s²/rpm²
ω → T (propeller)	c_T2	-2.881×10^{-4}	N·s/rpm
ω → Q (propeller)	c_Q1	7.120×10^{-9}	N·m·s²/rpm²
ω → Q (propeller)	c_Q2	-7.457×10^{-6}	N·m·s/rpm
PWM → T (integrated)	c_t1	-4.082×10^{-8}	N/μs³
PWM → T (integrated)	c_t2	1.841×10^{-4}	N/μs²
PWM → T (integrated)	c_t3	-0.2486	N/μs
PWM → Q (integrated)	c_q1	-7.95×10^{-10}	N·m/μs³
PWM → Q (integrated)	c_q2	3.619×10^{-6}	N·m/μs²
PWM → Q (integrated)	c_q3	-4.910×10^{-3}	N·m/μs

A.2 Dynamic Propulsion Transfer Functions — FX450

TF Model	Numerator Polynomial	Denominator Polynomial	Order	Fit	Valid BW
ω_s → ω_d	10.18s − 9.326	s² + 8.014s − 9.268	2nd	82.2%	0–1 Hz

TF Model	Numerator Polynomial	Denominator Polynomial	Order	Fit	Valid BW
T_s → T_d	$0.1447s^4 + 79.97s^3 + 5381s^2 + 1.17e6s + 3.689e6$	$s^4 + 132.2s^3 + 1.291e4s^2 + 1.303e6s + 3.847e6$	4th	94.6%	0–20 Hz
Q_s → Q_d	0.986	1 (static gain)	0th	~68%	0–1 Hz

A.3 Structural Parameters — FX450 / FX200

Parameter	Symbol	FX450	FX200	Unit
Mass	m	2.015	1.019	kg
Roll MOI	J_x	0.01850	—	$kg·m^2$
Pitch MOI	J_y	0.02030	—	$kg·m^2$
Yaw MOI	J_z	0.03223	—	$kg·m^2$
Roll damping	C_dp	0.0064	—	s^{-1}
Pitch damping	C_dq	0.0128	—	s^{-1}
Yaw damping	C_dr	0.0120	—	s^{-1}
Aero drag (X)	C_dx	—	0.40 (indoor) / 0.43 (outdoor)	N·s/m
Corrected thrust slope	a_T	—	3.847	N/normalized PWM
Corrected thrust intercept	b_T	—	1.459	N

Note: FX450 values (MOI, damping) are from the bifilar pendulum experiment. FX200 drag and thrust correction values are from real-flight identification. '—' indicates value not specifically identified for that model in this study.

A.4 Frequency-Domain Model — FX200 Pitch Channel

Model Type	Transfer Function	Order	Fit (%)	Valid Range
Open-loop (pitch)	Not recommended (max fit 43.5%)	5th	43.5%	< 35 rad/s
Closed-loop (pitch)	$(0.08012s^3 - 10.43s^2 + 1056s + 9234) / (s^3 + 34.16s^2 + 820.9s + 9635)$	3rd	85.5%	Up to ~30 Hz

A.5 Key Model Applicability Summary

Application	Recommended Model(s)	Limitation
Controller feasibility test (HITL)	Corrected 6-DOF model: Eq.(21) with a_T, b_T from Stage 3	Valid for low-to-medium speed; not validated for high-maneuver (>30 deg/s attitude rate)
Frequency-domain controller design	Closed-loop TF, Eq.(33)	Valid for pitch channel only; roll/yaw require separate identification
Propulsion simulation (0–20 Hz)	Two-stage dynamic TF: T_s→T_d (4th order)	Yaw torque TF poorly identified; use static model for Q

Application	Recommended Model(s)	Limitation
Cluster / formation flight disturbance	Disturbance TF model (Stage 5)	Not yet fully validated for multi-UAV wake scenarios
Fault simulation	Full 6-DOF model with modified propulsion parameters	Requires additional experimental validation per fault type

Appendix B Model Scope, Assumptions and Limitations

B.1 Model Scope

The time-domain and frequency-domain models presented in this manual are identified from experiments with the Zhuoyi FX-series quadrotor UAVs (FX150, FX200, FX310, FX450) under the following conditions:

- Flight speed: low to medium (typically 0–10 m/s forward flight)
- Attitude rates: non-aggressive (pitch/roll < 30 deg/s, yaw < 50 deg/s)
- Altitude: above ground-effect regime (> 1.5× propeller diameter)
- Wind conditions: light wind to calm (outdoor tests conducted at < 6 m/s ambient wind)

B.2 Model Assumptions

The following assumptions underpin the model structure and should be verified before applying the model to a different platform:

1. UAV is a rigid, symmetric body — arm flex, vibration, and CG offset are neglected.
2. Flight controller is installed at the geometric center with negligible installation tilt. If significant tilt exists, a correction term must be added.
3. Propeller thrust and torque coefficients are identical across all four motors/propellers (symmetric configuration). Replace with individual coefficients for asymmetric setups.
4. The drag model is isotropic in the horizontal plane (C_dx = C_dy). If the UAV is aerodynamically asymmetric, identify each axis independently.
5. Gyroscopic effects from motor rotation are negligible. For high-speed motors or aggressive maneuvers, a gyroscopic term may be required.

B.3 Known Limitations

Limitation	Impact	Recommended Action
Linear thrust model (Stage 3)	Approximation error at extreme PWM values	Acceptable for normal flight; re-identify if wide PWM range required
Static gain torque model	Poor yaw HITL fidelity	Use torque as TF input (not speed) for better Q identification

Limitation	Impact	Recommended Action
No ground-effect model	Z-axis PWM prediction deviates near ground	Add ground-effect correction term for near-ground operation
No structural flexibility	High-frequency vibration not captured	Add vibrational modes if operating near resonance frequencies
Wind disturbance not modeled	Outdoor HITL fidelity reduced in crosswind	Add wind input model for outdoor simulation fidelity
Open-loop freq-domain model poor fit	Cannot use open-loop TF for controller design	Use closed-loop TF (85% fit) as the frequency-domain model

Appendix C Worked Example: Building the FX200 6-DOF Simulation Model

This appendix walks through the complete assembly of the FX200 time-domain simulation model using the identified parameters from Stages 1–3. It serves as a practical guide for reproducing the HITL simulation described in Section 5.

C.1 Parameter Assembly

Collect all identified parameters into a single model parameter structure:

```
% ── FX200 Identified Model Parameters ──────────────────────────────────
% Structural parameters
m   = 1.019;          % mass [kg]
g   = 9.8;            % gravitational acceleration [m/s²]
l   = 0.155;          % arm length [m]  (FX200 wheelbase / 2)

% Corrected thrust model (Stage 3 result)
a_T = 3.847;          % PWM-thrust slope  [N / normalized-PWM-unit]
b_T = 1.459;          % PWM-thrust intercept  [N]
a_M = 0.0203;         % reaction-torque slope (from static Stage 1 model)

% Drag coefficients (Stage 2, indoor identification)
C_dx = 0.40;          % X-axis viscous drag [N·s/m]
C_dy = 0.40;          % Y-axis viscous drag [N·s/m]  (assumed = C_dx)
C_dz = 0.45;          % Z-axis viscous drag [N·s/m]  (slightly higher)

% Rotational damping (Stage 2, bifilar pendulum; small → often set to 0)
C_dp = 0.0064;   C_dq = 0.0128;   C_dr = 0.0120;   % [s⁻¹]

% Moments of inertia (Stage 2, bifilar pendulum)
Jx = 0.00780;  Jy = 0.00810;  Jz = 0.01050;  % [kg·m²]  FX200 estimates
```

C.2 Equations of Motion — MATLAB/Simulink Implementation

The continuous-time equations of motion are implemented as a state-space ODE. For a Simulink implementation, the following MATLAB function block computes state

derivatives:

```
function dx = uav_eom(t, x, u, params)
% State vector x = [vx; vy; vz; p; q; r; phi; theta; psi]
% Input vector u = [sigma1; sigma2; sigma3; sigma4]  (PWM each motor)

vx=x(1); vy=x(2); vz=x(3);
p=x(4);  q=x(5);  r=x(6);
phi=x(7); theta=x(8);

% Compute individual thrust and torque
Ti = params.a_T * u + params.b_T;   % [4×1 thrust vector, N]
Qi = params.a_M * u;                 % [4×1 torque vector, N·m]
T  = sum(Ti);                         % total thrust

% Torque allocation (X-type quadrotor)
tau_p = (sqrt(2)*params.l/2)*(-Ti(1)+Ti(2)+Ti(3)-Ti(4));
tau_q = (sqrt(2)*params.l/2)*( Ti(1)-Ti(2)+Ti(3)-Ti(4));
tau_r = Qi(1)+Qi(2)-Qi(3)-Qi(4);

% Translational dynamics (viscous drag)
dvx = (T*sin(theta)          - params.C_dx*vx) / params.m;
dvy = (T*sin(phi)*cos(theta) - params.C_dy*vy) / params.m;
dvz = (T*cos(phi)*cos(theta) - params.m*params.g - params.C_dz*vz)/params.m;

% Rotational dynamics
dp = (tau_p - params.C_dp*p) / params.Jx;
dq = (tau_q - params.C_dq*q) / params.Jy;
dr = (tau_r - params.C_dr*r) / params.Jz;

% Euler angle kinematics (simplified small-angle not used here)
dphi   = p + (q*sin(phi)+r*cos(phi))*tan(theta);
dtheta = q*cos(phi) - r*sin(phi);
dpsi   = (q*sin(phi)+r*cos(phi)) / cos(theta);

dx = [dvx; dvy; dvz; dp; dq; dr; dphi; dtheta; dpsi];
end
```

C.3 Propulsion Model Integration — Dynamic Thrust Filter

To incorporate the dynamic thrust filter G_T(s) into the simulation, the static thrust T_s is first computed from the PWM input, then passed through the 4th-order TF to obtain the dynamic thrust T_d:

```
% In continuous simulation, implement G_T(s) as a state-space filter
% G_T(s) = (0.1447s^4 + 79.97s^3 + 5381s^2 + 1.17e6s + 3.689e6)
%          / (s^4 + 132.2s^3 + 1.291e4s^2 + 1.303e6s + 3.847e6)

sys_GT = tf([0.1447 79.97 5381 1.17e6 3.689e6], ...
            [1 132.2 1.291e4 1.303e6 3.847e6]);
[A_GT, B_GT, C_GT, D_GT] = ssdata(sys_GT);  % convert to state-space

% For each propeller, chain: PWM → T_static → T_dynamic
% Ts_i = a_T*sigma_i + b_T   (computed each timestep)
% Td_i = lsim(sys_GT, Ts_i, t_vec)   (or integrate SS in ODE loop)
```

Note: *For computationally efficient HITL simulation, the 4th-order filter can be approximated by a*

lower-order Padé approximation or a simple 1st-order lag if the target bandwidth is below 2 Hz.

C.4 Model Validation Quick Check

Before running the full HITL experiment, perform these sanity checks on the assembled model:

Check	Expected Outcome	If Failed
Hover thrust balance	At hover PWM ≈ 1480 μs per motor, T_total ≈ m·g = 9.99 N	Recheck a_T, b_T identification
Step velocity overshoot	<30% for 0.5 m/s step with MCC controller	Reduce bandwidth of MCC controller
Yaw rate response	Smooth with <20% overshoot	Check a_M; may need adjustment from static Q model
Z-axis altitude hold	Oscillation < 0.05 m peak in still air	Increase Z-axis damping (C_dz) or reduce integral gain

Appendix D Dynamic Thrust Under Inflow — Advance Ratio Effects

D.1 Physical Mechanism

When the UAV moves forward at speed V, the propellers experience an inflow with velocity component V·sinα (where α is the angle between the propeller plane and the inflow direction). This inflow reduces the effective angle-of-attack of the blade sections, decreasing both thrust and reaction torque — a phenomenon characterized by the advance ratio:

$$J = V / (n \cdot D) \quad \text{where } n = \text{rotational speed [rev/s]}, D = \text{propeller diameter [m]} \tag{A1}$$

For a fixed-pitch propeller, increasing J reduces both the thrust coefficient C_T and torque coefficient C_Q. This effect is captured in the wind tunnel dynamic thrust measurements described in Section 2.2.2 of the complete study.

D.2 Wind Tunnel Measurement Results — FX310

Wind tunnel measurements were conducted at eight wind levels (corresponding to 0–~12 m/s), seven throttle settings (PWM 1100–1700 μs), and angles of attack from 0° to 90° (vertical climb to horizontal forward flight). Key findings:

- Under the same PWM input, propeller thrust decreases with increasing inflow speed. Attenuation is significant at wind level 5 and above (~7 m/s+).
- Rotational speed changes are small at levels 1–4 but decrease measurably at levels 5–8, despite thrust showing earlier attenuation.

- At levels 3–4, an anomalous observation was noted: rotational speed increased slightly while thrust still decreased — indicating a change in blade pitch effectiveness.
- Reaction torque attenuation follows a qualitatively similar trend to thrust but with a different quantitative relationship.

Dynamic thrust model parameters under inflow (FX310, rotational speed → thrust model: $T = C_T0 \cdot \omega^2 + C_T1 \cdot \omega$):

Wind Level	Approx. Speed (m/s)	C_T0 ($\times 10^{-8}$)	C_T1 ($\times 10^{-4}$)	Thrust Change at PWM=1400
1 (calm)	~1.5	8.568	−1.299	Baseline
2	~2.8	8.196	−0.875	-3% vs calm
3	~4.2	8.510	−1.582	-5% vs calm
4	~5.5	8.123	−2.877	-10% vs calm
5	~6.8	9.100	−2.818	-15% vs calm
6	~8.0	8.957	−3.100	-22% vs calm
7	~9.5	9.372	−3.779	-30% vs calm
8	~11.0	9.689	−4.437	-38% vs calm

D.3 Decoupled Modeling Strategy

Rather than incorporating inflow effects directly into the propulsion polynomial, this study adopts a decoupled strategy: the advance ratio effect is uniformly modeled as an equivalent aerodynamic drag force in the body axis. This approach is justified because:

6. Thrust loss due to inflow = equivalent drag force opposing forward motion → naturally captured by the viscous drag model.
7. This avoids coupling inflow speed (a global state variable) into the per-motor propulsion model, greatly reducing model complexity.
8. Wind tunnel validation confirmed that the identified drag coefficient C_dx accounts for the net thrust attenuation observable as forward-flight resistance.

The consequence: the propulsion model ($T_i = a_T \cdot \sigma_i + b_T$) remains valid as the vacuum thrust at the motor's operating point, and the translational dynamics ($m \cdot \dot{v}_x = T \cdot \sin\theta - C_dx \cdot v_x$) implicitly absorbs the advance ratio effect through the drag term. This architecture is validated through real-flight HITL comparison in Section 5.

Appendix E Data Processing Reference

E.1 Flight Log Processing Pipeline

Raw flight logs (in PX4 .ulg format) must be converted and processed before

identification. The following pipeline applies to all real-flight identification experiments in this manual.

```
% Step 1: Convert .ulg flight log to .csv tables
% ulog2csv.m  (Rflysim platform utility)
% Produces one .csv per topic, e.g.:
%   vehicle_local_position_0.csv   → velocity, position
%   vehicle_angular_velocity_0.csv → p, q, r
%   actuator_outputs_0.csv          → PWM per channel
%   vehicle_attitude_0.csv          → quaternion / Euler angles

% Step 2: Import required columns into MATLAB workspace
% Example: import X-axis velocity
data = readmatrix('vehicle_local_position_0.csv');
t    = data(:, col_timestamp) / 1e6;   % µs → seconds
vx   = data(:, col_vx);                % m/s

% Step 3: Align timestamps across topics
% All topics use the same flight controller clock (µs since boot)
% Interpolate to common time vector if sampling rates differ
t_common = linspace(max([t1(1) t2(1)]), min([t1(end) t2(end)]), N);
vx_sync  = interp1(t1, vx, t_common, 'linear');
pitch_sync = interp1(t2, pitch, t_common, 'linear');
```

E.2 De-meaning for Drag Identification

Before drag identification, the control output must be de-meaned to remove steady-state biases caused by CG offset, flight-controller installation tilt, and any constant wind. The datapreproc function from the Rflysim package performs this:

```
% De-mean the X-axis control force
ux_raw = m * g * sin(pitch/g) ./ (cos(roll) .* cos(pitch));
ux_dm  = datapreproc(ux_raw(t_start:t_end), 0.05, ...
                     'rmean', true, 'rtrend', false, 'tap_width', 0);
% Arguments:
%   0.05       : sample time [s]
%   'rmean'    : remove mean (true = apply de-meaning)
%   'rtrend'   : remove linear trend (false = not applied)
%   'tap_width': taper window width for edge effects

% Alternative manual de-meaning:
ux_dm = ux_raw - mean(ux_raw);
```

E.3 Frequency Response Function Estimation Settings

The spafdr function from the MATLAB System Identification Toolbox estimates the empirical frequency response function (FRF). Key settings affect estimation quality:

Setting	Recommended Value	Effect of Increasing	Effect of Decreasing
Frequency vector resolution	500 points, linear spacing	Smoother FRF; slower computation	Coarser FRF; faster
Data window length	Full sweep duration	Better low-freq resolution	Better high-freq resolution
Spectral smoothing	Automatic in spafdr	Smoother FRF; less noise	Noisy FRF

Setting	Recommended Value	Effect of Increasing	Effect of Decreasing
(default)			
Log vs. linear spacing	Linear for TF fit; log for Bode	Better low-freq coverage (log)	Better high-freq coverage (linear)

E.4 Transfer Function Model Order Selection

The tfest function fits a transfer function to the empirical FRF. The selection of model order involves a trade-off between fit quality and model complexity:

Order	Typical Fit Range	Recommended Use	Caution
1st	40–65%	First-pass drag identification (viscous form)	May miss resonance dynamics
2nd	60–82%	Motor dynamic model (two-stage ω_s→ω_d)	Check for unstable poles
3rd	75–86%	Closed-loop attitude TF; drag with delay	Verify physical interpretability
4th	85–95%	Dynamic thrust TF (T_s→T_d)	Computational cost in simulation
5th+	85–98%	Only if 4th-order insufficient	High risk of overfitting; avoid

```
% Systematic order selection
for n = 1:6
    sys_n    = tfest(g_frf, n);
    [~, fit_n] = compare(sys_n, id_data);
    fit_vec(n) = fit_n;
    fprintf('Order %d: fit = %.2f%% ', n, fit_n);
end
% Select lowest order with fit > threshold (e.g., 80%)
n_sel = find(fit_vec >= 80, 1, 'first');
if isempty(n_sel), n_sel = find(fit_vec == max(fit_vec), 1); end
sys_final = tfest(g_frf, n_sel);
disp(['Selected order: ', num2str(n_sel), ', fit: ', num2str(fit_vec(n_sel)),
'%']);
```

E.5 Common Issues and Troubleshooting

Issue	Likely Cause	Resolution
Very low fit (<40%) at all orders	Insufficient excitation signal; poor signal-to-noise ratio	Increase sweep amplitude; ensure CH7 sweep is correctly injected
High fit in time domain but poor Bode match	Overfitting to noise; model order too high	Reduce model order; use logspace frequency vector for Bode focus
Unstable poles in identified TF	Noisy data at edges of frequency range; numerical issues	Trim data to clean sweep segment; add stability constraint in tfest options
Downsampling	resample() edge transients	Discard first and last 5% of downsampled data before

Issue	Likely Cause	Resolution
artifacts (front/end of data)		identification
spafdr returns flat FRF	Data not properly filtered; aliasing	Lower lpf cut-off; verify downsample rate = 20×f_max
lsqcurvefit does not converge	Poor initial guess; function poorly scaled	Try multiple starting points; normalize inputs (e.g., PWM/1000)

Appendix F Relationship Between Model Components

The diagram below summarizes how all identified model components integrate into the final simulation:

Input Signal	Processing Step	Output Signal	Used In
PWM σ_i (μs)	Static motor model: $\omega_s = c_w1 \cdot \sigma^2 + c_w2 \cdot \sigma - offset$	Static speed ω_s (rpm)	Dynamic speed TF input
Static speed ω_s	Dynamic TF G_ω(s): ω_s→ω_d	Dynamic speed ω_d (rpm)	Propeller static model input
Dynamic speed ω_d	Static propeller model: $T_s = c_T1 \cdot \omega^2 + c_T2 \cdot \omega$	Static thrust T_s (N)	Dynamic thrust TF input
Static thrust T_s	Dynamic TF G_T(s): T_s→T_d	Dynamic thrust T_d (N)	6-DOF equations of motion
PWM σ_i (directly)	Corrected thrust: $T_i = a_T \cdot \sigma_i + b_T$ (Stage 3)	Individual thrust (N)	6-DOF equations (simplified path)
Thrust T_i per motor	Allocation matrix (Eq.2)	τ_p, τ_q, τ_r (N·m)	Rotational dynamics
Total thrust $T = \Sigma T_i$	6-DOF translational equations	$\dot{v}_x$, $\dot{v}_y$, $\dot{v}_z$ (m/s²)	State integration
Angular rates p, q, r	Rotational dynamics with drag (Eq.21)	$\dot{p}$, $\dot{q}$, $\dot{r}$ (rad/s²)	State integration
Attitude φ, θ, ψ	Euler kinematics	$\dot{\varphi}$, $\dot{\theta}$, $\dot{\psi}$ (rad/s)	State integration

Note: In the corrected model (Stage 3), the multi-stage dynamic chain (PWM→ω→T_s→T_d) is replaced by the direct linear map $T_i = a_T \cdot \sigma_i + b_T$. This simplified path is used for HITL simulation because it is more consistent with in-flight propulsion behavior. The full dynamic chain is recommended when precise frequency-domain propulsion characterization is required.

—— *End of Document* ——